Multifeature Collaborative Attention Dynamic Hypergraph Convolutional Network for Hyperspectral Image Classification

Wenping Liu, Yuxiang Zhang, *Senior Member, IEEE*, and Yanni Dong, *Senior Member, IEEE*

***Abstract*—Most of the current hyperspectral image classification (HSIC) methods assume that the interactions among all ground objects in hyperspectral images (HSIs) are static pairwise relationships. However, in real scenarios, multiple ground objects have complex spatial, spectral, or statistical correlations. These correlations are not limited to simple adjacent or pairwise relationships but also include complex higher order interactions involving three or more ground object categories. A hyperedge in a hypergraph can simultaneously connect multiple vertices, effectively capturing the multi-dimensional and high-order relationships among vertices. To address the limitations of the current mainstream methods in modeling the high-order interaction relationships of ground objects, a novel multifeature collaborative attention dynamic hypergraph convolutional (MDHGC) network is proposed to model the entire HSI and capture the high-order relationships among ground objects, thereby achieving accurate classification. Specifically, we designed a static–dynamic collaborative multiview hypergraph convolutional network based on differential attention to learn superpixel-level features, which allows stable and flexible learning of high-order interactions in HSI. To learn the complementary features of pixel-level HSIC, we introduce a branch based on convolutional neural neworks that includes multiscale feature extraction and global–local feature fusion. Comprehensive experiments have been conducted across four distinct datasets to rigorously evaluate the effectiveness of MDHGC.**

***Index Terms*—Attention mechanism, convolutional neural networks (CNNs), hypergraph convolutional networks, hyperspectral image classification (HSIC).**

I. Introduction

HYPERSPECTRAL sensors can capture the spectral characteristics of ground objects across hundreds of continuous narrow bands in the visible to near-infrared wavelength range and simultaneously obtain rich spatial features. Therefore, hyperspectral images (HSIs) have applications in various fields, such as geological exploration [1], disaster warning [2], agricultural monitoring [3], and medical diagnosis [4].

Hyperspectral image classification (HSIC) categorizes each pixel in HSI to a specific land cover class based on its characteristics [5]. HSIC has long been a focal point of research in the field of remote sensing [6]. Early HSIC methods primarily relied on spectral feature matching [7], such as spectral angle mapping [8], and statistical models, such as maximum likelihood classification [9]. These methods are simple and fast but are highly influenced by human factors. With advancements in technology, machine learning algorithms, which are an artificial intelligence technique that enhances statistical learning algorithms by incorporating domain expertise, have also been applied to HSIC. The classification process first extracts discriminative features from HSI using methodologies like linear discriminant analysis (LDA) [10], band selection [11], and gray-level co-occurrence matrix (GLCM) [12]. These techniques effectively address the inherent redundancy and complexity of high-dimensional datasets [13]. Subsequently, machine learning models like support vector machine (SVM) [14] and ensemble learning algorithms [15] are trained and employed as classifiers to achieve optimal pattern recognition performance. However, as the number of spectral bands in HSI gradually increases, machine learning models can only utilize shallow features [16] and cannot accurately identify the mapping relations between features and categories.

In the past decade, deep learning algorithms have become the frontier application and research focus in HSIC [17]. Compared with the shallow feature extraction of machine learning, deep learning can extract diverse and abstract structured features layer by layer [18]. Then, the SoftMax function is utilized to achieve the classification [19]. Convolutional neural networks (CNNs) extract local features through convolution operations, reduce the number of parameters, and improve computational efficiency through weight sharing, making them the most widely used method for feature extraction [20]. Subsequently, various CNNs architectures have been introduced, such as 2D-CNN [21], 3D-CNN [22], and hybrid CNN [23], to increase the spectral–spatial expression ability. Zhong et al. [24] used 3D-CNN and residual structures (SSRN) to continuously extract the spatial and spectral information of HSI. Zhang et al. [25] introduced a spectral partition residual network (SPRN), which used grouped convolutions instead of dimensionality reduction to address the issue of

Received 26 May 2025; revised 25 June 2025 and 28 July 2025; accepted 10 August 2025. Date of publication 13 August 2025; date of current version 28 August 2025. This study was jointly supported by the National Natural Science Foundation of China under Grant 62222116 and Grant 62171417 and in part by the Key Science and Technology Project of the Ministry of Emergency Management of People's Republic of China under Grant 2024EMST070706. *(Corresponding author: Yanni Dong.)*

Wenping Liu is with the School of Geophysics and Geomatics, China University of Geosciences, Wuhan 430074, China, and also with the School of Resource and Environmental Sciences, Wuhan University, Wuhan 430079, China (e-mail: 3221058401@qq.com).

Yuxiang Zhang is with the School of Geophysics and Geomatics, China University of Geosciences, Wuhan 430074, China (e-mail: zhangyx@cug.edu.cn).

Yanni Dong is with the School of Resource and Environmental Sciences, Wuhan University, Wuhan 430079, China (e-mail: dongyanni@whu.edu.cn).

Digital Object Identifier 10.1109/TGRS.2025.3598375

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:26:46 UTC from IEEE Xplore. Restrictions apply.

high dimensionality in HSI. Yang et al. [26] extracted and fused multiscale features for HSIC through CNN (EMFFN). However, CNN still faces some challenges in HSIC. On the one hand, due to the limitation of Euclidean data structure, it is difficult for CNN to model the complex topological relationships in the high-dimensional data space. On the other hand, due to the restriction of fixed convolution kernel size, CNN cannot capture global information and cannot handle the dependencies between pixels over long distances.

To make up for the aforementioned deficiencies, graph convolution networks (GCNs) have received extensive attention. GCNs, from a non-Euclidean perspective, learn feature representations by leveraging the information of adjacent vertices on graph structures, enabling rapid information propagation between vertices and capturing long-range features. Qin et al. [27] first proposed an HSIC network based on GCN—the spectral–spatial GCN (S2GCN). However, S2GCN treats each pixel in the HSI as a graph vertex, leading to a significant increase in computational cost. To address this issue, one solution is to use a mini-batch strategy, where only a subset of vertices and edges is loaded for training at each iteration. For instance, Hong et al. [28] constructed a mini-batch GCN that enables HSI training in small batches. However, this method might overlook the global context and lose full-graph information. Another approach is to optimize the graph structure, such as constructing a sparse graph or using superpixel vertices instead of pixel vertices. For instance, Li et al. [29] constructed adjacency graphs and sparse graph structures at different stages, effectively improving computational efficiency. However, sparsification might lead to information loss, making it difficult to model long-range dependencies. Superpixel segmentation methods are the current main solution, as they preserve the essential structural information and overcome the constraints of the original Euclidean data structure. Wan et al. [30] first segment HSI into a series of superpixels, significantly reducing the number of vertices. Lu and Kamata [31] built a multilevel superpixel graph structure through the entropy rate superpixel segmentation (ERS) [32] algorithm, which effectively improved the accuracy of HSIC.

CNNs excel at processing local information and fine-grained features in images but struggle with modeling global information. GCNs can consider the global graph structure and capture complex relationships between vertices, but their computational cost is relatively high. Although superpixel-based graph methods significantly reduce computational complexity, the superpixel segmentation process inevitably leads to the loss of internal details, which is unfavorable for the classification of fine-grained objects. By combining both approaches, hybrid networks can leverage both local features from the image and graph structural information, thereby improving the ability to recognize land cover types. For instance, Liu et al. [33] integrated CNNs and GCNs (CEGCN) to extract ensemble features. Dong et al. [34] combined graph attention networks and CNNs with an attention mechanism (WFCG) to enhance HSIC. Zhou et al. [35] integrated multiscale CNNs and multihop GCNs with attention fusion (AMGCFN) modules to effectively extract multilevel features.

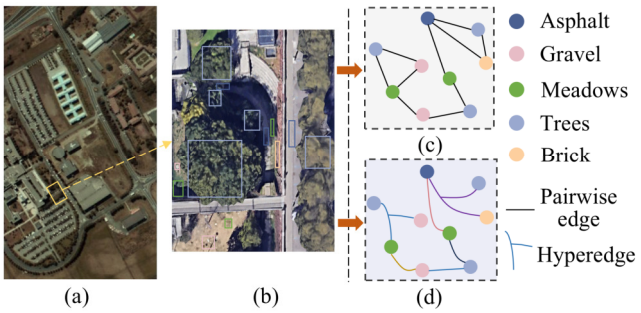


Fig. 1. Differences between hypergraphs and simple graphs. (a) University of Pavia dataset. (b) Input example using a city scenario. (c) Simple graph: each edge connects exactly two vertices, representing pairwise relationships only. (d) Hypergraph: each hyperedge can connect multiple vertices, enabling the modeling of higher order interaction relationships.

Although the aforementioned studies have demonstrated the effectiveness of hybrid networks, there are still some issues that urgently need to be addressed. In real life, multiple land cover categories interact with each other through nonlinear or synergistic effects, and such interactions cannot be explained by the simple superposition of single elements or pairwise relationships. As shown in Fig. 1(a), taking the University of Pavia dataset as an example, in urban scenarios, asphalt (main roads) is usually adjacent to bricks (sidewalks), and they are separated by green belts (meadows or trees), while newly planted trees are often surrounded by asphalt (boundaries of roads) and bare gravel or soil. This synergistic landscape adjacency, where asphalt, bricks, and trees form a "road-greening" nested structure, requires the simultaneous modeling of multiple types of relationships. However, previous studies typically rely on simple graphs, which model the vertices and edges based on the objects and their pairwise relationships. This approach fails to adequately capture the complex higher order interactions among multiple objects, which significantly impacts the effectiveness of feature representation [36].

Unlike simple graphs, which only model low-order pairwise interactions between vertices [Fig. 1(c)], a hypergraph is an extension of a simple graph [37]. Its hyperedges can connect any number of vertices, allowing it to model not only pairwise relationships but also complex higher order interactions. As shown in Fig. 1(d), the number of vertices connected by a hyperedge can also be adjusted flexibly depending on the complexity of the actual interactions, preserving the overall information of the objects rather than indirectly representing them by decomposing multiple pairwise relationships.

Feng et al. [38] were the first to propose hypergraph neural networks (HGNNs) and applied them to the learning and representation of graph-structured data. Their work demonstrated that HGNNs are effective in capturing higher order relationships between vertices, which is crucial for complex data structures in many practical applications. Currently, research on HSI classification based on HGNNs is relatively scarce. Ma et al. [39] proposed a pixel-level hypergraph convolutional network (HGCN) that integrates the spatial, spectral, and CNN features of HSI (F2HNN). Duan et al. [20] constructed a structure-preserved hypergraph convolutional network (SPHGCN), which constructed a partial pixel hypergraph and extracted cross-patch structural features for

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:26:46 UTC from IEEE Xplore. Restrictions apply.

HSIC. Wang et al. [40] integrated CNN and HGCN and proposed a hypergraph convolutional network with multiple hyperedges fusion (HGCN-MHF). Wang et al. [41] also combined spectral–spatial hypergraph convolutional networks and spectral–spatial convolutional networks (DF2Net).

However, most HGNN-based HSIC methods still have some limitations.

1) They use a fixed hypergraph structure to extract features, which ignores dynamically changing topological structure information.
2) The hypergraph structure is constructed directly on superpixels or a small number of pixels, which limits the representation of key features.

Building on the preceding analysis, a novel multifeature collaborative attention dynamic hypergraph convolutional (MDHGC) network for HSIC is proposed. First, we design a superpixel-level branch that constructs multiview hypergraphs from spectral and spatial directions to more fully represent complex interactions in HSI. Specifically, in order to help hypergraph convolution (HGC) focus on key regions and replenish missing detail features, we proposed a vertex differential attention module (VDAM) and a feature differential attention module (FDAM) to calculate the correlation between graph vertices and their features. To improve the model's stability and adaptability across various tasks, we design a static–dynamic collaborative multiview hypergraph convolutional network based on differential attention to extract superpixel features. To enhance the model's robustness when applied to different datasets, we propose an adaptive adjustment simple linear iterative clustering (A2SLIC) method that reduces dependence on the segmentation scale. Next, to supplement the internal details lost in the process of superpixel segmentation, we design a pixel-level branch, which mainly extracts the fine-grained target features of HSI by CNN. Specifically, we construct a multiscale feature extraction (MSFE) module that can fuse more discriminative high-level and low-level features to generate complementary feature representations for HSIC. By combining CNN and the self-attention mechanism, we propose a global–local fusion module (GLFM) to consider both local and global information to enhance classification performance. Finally, the extracted superpixel-level and pixel-level complementary features are fused in a weighted way.

The principal breakthroughs achieved in this research are as follows.

1) To fully exploit the pairwise and high-order interaction relationships in HSI to obtain precise HSIC, we propose a superpixel-level branch based on a static–dynamic collaborative multiview hypergraph convolutional network from different perspectives to capture superpixel-level features.
2) To supplement the lost internal detailed features during superpixel segmentation, we design a pixel-level branch based on an MSFE and a GLFM to fully exploit the rich spectral and spatial pixel-level features and capture both long-range dependencies and local correlations.
3) Applying the differential attention modules (DAM), VDAM and FDAM, to allocate the attention weights for each graph vertex and its feature. Instead of aggregating all vertices equally, this helps the HGNN to better focus on the key areas and supplement the lost detailed features.

This article is organized as follows. Section II introduces the MDHGC, Section III presents experiment settings and results, Section IV analyzes the effects of different parameters, and Section V provides a conclusion for this article.

II. Proposed Method

A. Overall Framework

As illustrated in Fig. 2, MDHGC consists of two branches, which extract pixel-level and superpixel-level features. In this section, we denote the HSI by $\mathbf{X} \in \mathbb{R}^{H\times W\times B}$, where H is the height, W is the width, and B is the spectral dimension.

When extracting pixel-level features, the input HSI first undergoes an MSFE to effectively utilize the abundant spectral features and suppress noise in the HSI. Then, the GLFM captures both long-range dependencies and local spectral correlations. Subsequently, a 1×1 convolution operation is used to extract deep spectral features and adjust the channel dimensions of the features. A channel shuffle operation is then performed, where the input features are divided into multiple non-overlapping groups along the channel dimension. Wherein groups, a 3×3 depthwise convolution is used to enhance cross-channel interactions and extract spatial features. This process is repeated twice to obtain the final pixel-level features F_{pixel}.

When extracting superpixel-level features, the first step is superpixelization via the A2SLIC method. Then, we construct a spatial incidence matrix H_{spa} and a spectral incidence matrix H_{spe} between superpixel vertices and fuse two incidence matrices to obtain a multiview hypergraph $G_{\text{multi}} = \{V, E, W\}$ to more comprehensively express the nonlinear cautionary relationship in HSI, where G represents the hypergraph, V represents the vertex set, E represents the hyperedge set, and W represents the hyperedge weight matrices. Then, the hypergraph passes through VDAM and FDAM to better leverage the correlations between vertices and features. To avoid oversmoothing problems, we use a parallel approach to combine the static hypergraph and the dynamic hypergraph. Among them, the dynamic hypergraph convolutional network (DHGCN) reconstructs the distance hypergraph and similarity hypergraph to dynamically adjust the graph structure. After that, the Vertex Conv and Hyperedge Conv are used to transform the input features and dynamically extract complex high-order features. Finally, we apply a Decoder to transform superpixels to pixels, thereby obtaining the feature $F_{\text{superpixel}}$.

Finally, we perform a weighted fusion of the features F_{pixel} and $F_{\text{superpixel}}$. After fusion, an MLP is used to adjust the feature vector's channel length to match the number of categories. Then, a SoftMax layer is applied to achieve the classification task. The method utilizes cross entropy as the loss function for training

$$F_{\text{all}} = \gamma * F_{\text{pixel}} + (1-\gamma) * F_{\text{superpixel}}. \tag{1}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:26:46 UTC from IEEE Xplore. Restrictions apply.

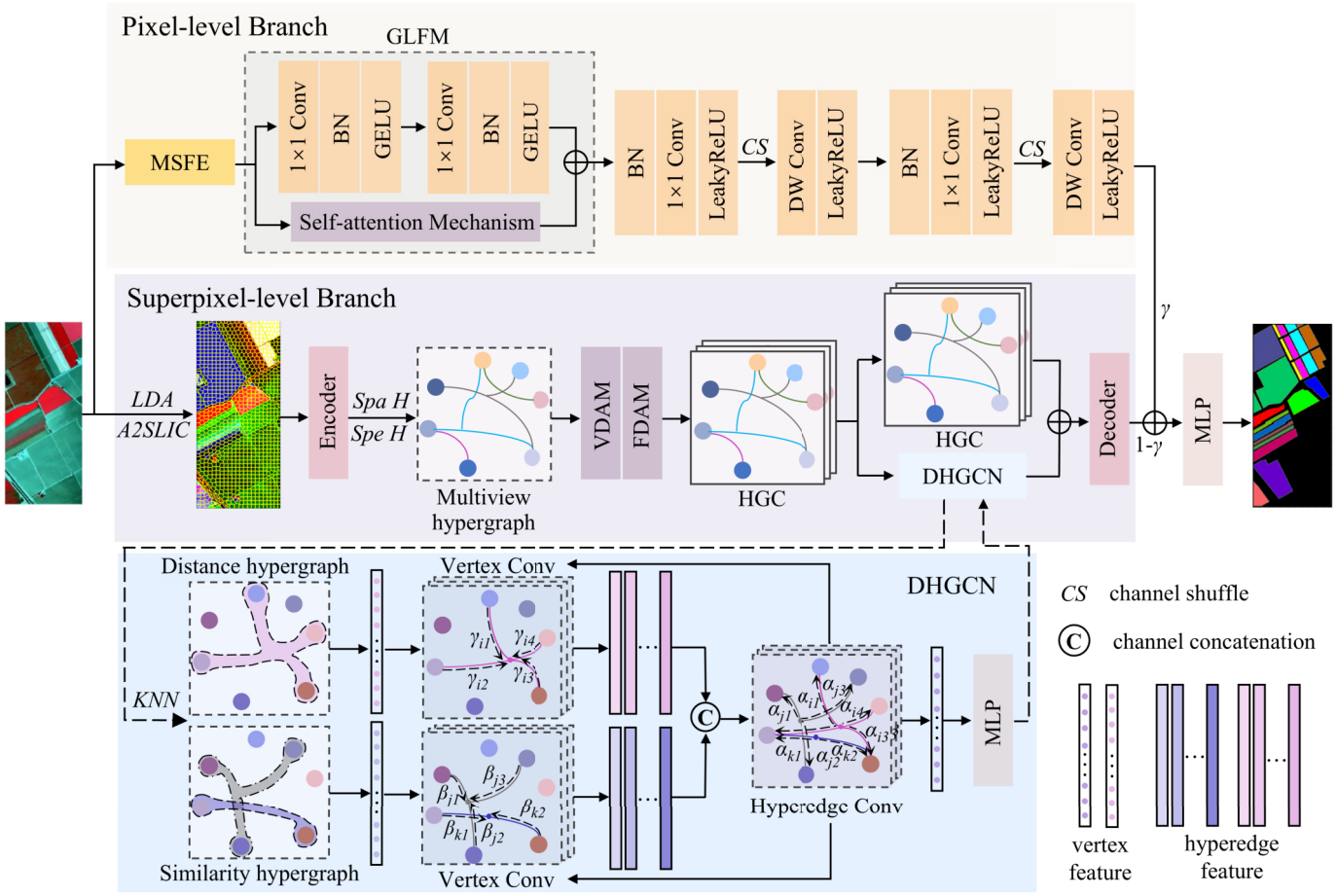


Fig. 2. Illustration of the proposed overall framework.

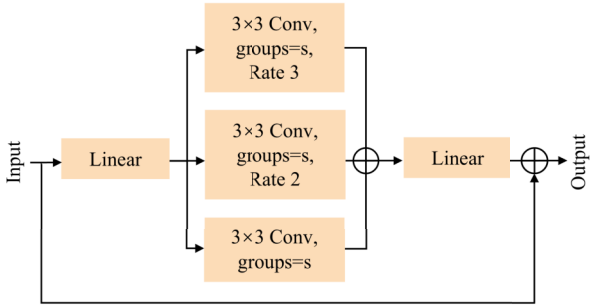


Fig. 3. Illustration of the multiscale feature extraction module.

B. Multiscale Feature Extraction Module

HSI contains numerous spectral bands, and a single spectral feature may not be sufficient to comprehensively represent the information in the HSI. Inspired by the wide application of pyramid convolution in the field of target detection, we propose the MSFE that can enhance the feature expression ability and robustness of the network. As shown in Fig. 3, a linear layer is first employed to expand the feature channels of the input HSI. Then, multiscale dilated convolutions are used on three parallel paths to extract features of different scales, with a cascading operation introduced to enhance the feature representation. Specifically, three 3×3 dilated depth-wise convolutions are utilized. Finally, the cascaded features are linearly projected to recover their original dimensionality, enabling a residual connection with the original features. Given a feature map $\hat{\mathbf{X}} \in \mathbb{R}^{h \times w \times b}$, MSFE is formulated as

$$\hat{\mathbf{X}}_{\text{gating}} = C_{3\times3}\left(W_1\hat{\mathbf{X}}\right) + C^2_{3\times3}\left(W_1\hat{\mathbf{X}}\right) + C^3_{3\times3}\left(W_1\hat{\mathbf{X}}\right) \quad (2)$$

$$\hat{\mathbf{X}}_{\text{out}} = W_2\hat{\mathbf{X}}_{\text{gating}} + \hat{\mathbf{X}} \quad (3)$$

where W_1 and W_2 denote linear layers; $C_{3\times3}$ denotes the 3×3 depth-wise convolution; and $C^2_{3\times3}$ and $C^3_{3\times3}$ denote dilated depth-wise convolution with dilation rate of 2 and 3, respectively.

C. Global–Local Fusion Module

Convolutional operations are constrained by the receptive field and local attention, often leading to an excessive focus on local features, which distorts sequence information. In contrast, the self-attention mechanism effectively models sequence information by establishing relationships between different pixels, enabling the network to fully learn global information. Therefore, we propose a GLFM that consists of parallel convolution and self-attention mechanisms to facilitate the simultaneous learning of both global and local features. The detailed structure is shown in Fig. 2. In the global branch, we directly use the input features as query (Q), key (K), and value (V) to conduct self-attention calculation to extract more global HSI features. In the local branch, two 1×1

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:26:46 UTC from IEEE Xplore. Restrictions apply.

convolution layers are used for feature extraction. Batch normalization (BN) layers adjust the data distribution of the features and reduce parameter initialization requirements to accelerate training. The GELU activation function is utilized to enhance the nonlinear transformation of the features. The local branch is formulated as

$$\hat{\mathbf{X}}^{l+1} = \phi\left(\mathbf{BN}\left(C_{1\times1}\left(\hat{\mathbf{X}}^{l}\right)\right)\right) \tag{4}$$

where $\hat{\mathbf{X}}^{\mathbf{l}}$ denotes the input feature of the lth layer, $\hat{\mathbf{X}}^{l+1}$ denotes the output feature of the lth layer, $C_{1\times1}$ denotes a 1×1 convolution, BN$(\cdot)$ represents a batch normalized layer, and $\phi(\cdot)$ represents the GELU nonlinear activation function. By applying (4) twice, the output of the local branch can be obtained.

D. Pixel-to-Superpixel Conversion Module

Directly treating the pixels of HSI $\mathbf{X} \in \mathbb{R}^{H\times W\times B}$ as vertices in a graph results in a massive graph and overly complex computations in subsequent steps. The simple linear iterative clustering (SLIC) method, known for generating smooth and compact superpixels with relatively simple usage and minimal parameter adjustments, remains the most widely applied superpixel segmentation approach to segment HSIs into superpixels $S \in \{1, 2, \ldots, Z\}$, where Z represents the number of superpixels, $Z = (H \times W)/\lambda$, and λ denotes the superpixel segmentation scale [42]. When constructing the graph using the SLIC algorithm, the segmentation scale λ deterministically influences how elementary pixels are aggregated into higher order superpixel vertices. A larger superpixel segmentation scale means that a superpixel contains more pixels. However, the SLIC method relies on a fixed segmentation scale that cannot adapt to different images. If the superpixel segmentation is not properly set, then the method of performing superpixel segmentation using the SLIC algorithm may limit its performance. Furthermore, the distance measurement of SLIC mainly relies on color and spatial position, and it makes insufficient use of direct information such as local texture and gradient.

To address this issue, we propose an A2SLIC method that comprehensively considers the edge strength (gradient magnitude) [43], texture complexity (local binary pattern, LBP) [44], and color variance [45] of the HSI to generate a complexity map

$$\mathrm{com} = 0.4^{\star}\mathrm{grad} + 0.4^{\star}\mathrm{lbp} + 0.2^{\star}\mathrm{var} \tag{5}$$

where *com* represents the complexity map of HSI, *grad* represents the edge strength, *lbp* represents the texture complexity, and *var* represents the color variance. The total number of superpixels Z is then adjusted adaptively according to this complexity map: the higher the overall complexity of the image, the more the total number of superpixels after segmentation (the smaller superpixel segmentation scale). Therefore, there is no need for manual adjustment of the superpixel segmentation scale. The A2SLIC method can adaptively adjust the segmentation according to the complexity of the input data.

Subsequently, K-means clustering is applied to group pixels with similar spatial and spectral information into superpixels. Then, we treat these superpixels as vertices in the graph to reduce computational complexity, where the feature of each vertex is represented by the average of the pixel features within the corresponding superpixel. Since the output of the features by the convolution branch and the final predicted labels are both pixel-level data, we use an association matrix $\mathbf{A} \in \mathbb{R}^{HW\times Z}$ to enable mutual transformation between pixels and superpixels. Then, we have

$$\mathbf{A}_{i,j} = \begin{cases} 1, & \text{if } \overline{\mathbf{X}}_i \in S_j \\ 0, & \text{otherwise} \end{cases} \quad \bar{\mathbf{X}} = Flatten\left(\mathbf{X}\right) \tag{6}$$

where $\bar{\mathbf{X}}$ represents the HSI data after flattening, $\bar{\mathbf{X}}_i$ represents the ith pixel in $\bar{\mathbf{X}}$, and S_j represents the jth superpixel.

Then, we can achieve the conversion between pixels and superpixels by using encoders and decoders as follows:

$$\mathbf{S} = \mathrm{Encoder}\left(\mathbf{X}; \mathbf{A}\right) = \hat{\mathbf{A}}^{T} Flatten\left(\mathbf{X}\right) \tag{7}$$

$$\mathbf{X}_{\mathrm{new}} = \mathrm{Decoder}\left(\mathbf{S}; \mathbf{A}\right) = Reshape\left(\mathbf{AS}\right) \tag{8}$$

where $\hat{\mathbf{A}}$ represents the normalized $\mathbf{A}$. $\mathbf{S}$ represents the feature matrix of all superpixel vertices, $Reshape(\cdot)$ refers to recovering the original size of the flattened data, and $\mathbf{X}_{\mathrm{new}}$ represents the pixel features derived from the superpixel features after the hypergraph convolution operation.

E. Hypergraph Construction

The hypergraph is a generalization of a relational structure, where a hyperedge can link multiple vertices, allowing for the learning of higher order interactions. Before extracting features using HGCN, it is crucial to construct a hypergraph to explore the higher order dependencies in the HSI. As described earlier, unlike simple graph data represented by an adjacency matrix, a hypergraph is represented by an incidence matrix $H \in \mathbb{R}^{|M|\times|N|}$, where $|M|$ is the number of vertices and $|N|$ is the number of hyperedges. Specifically

$$H_{m,n} = \begin{cases} 1, & v_m \in e_n \\ 0, & v_m \notin e_n \end{cases} \tag{9}$$

where v_m denotes the mth vertex in $\mathbf{V}$, e_n denotes the nth hyperedge in $\mathbf{E}$, and $v_m \in e_n$ denotes the vertex v_m connected by the hyperedge e_n. Once the hyperedge weights $W \in \mathbb{R}^{|N|\times|N|}$ and the incidence matrix H are obtained, we can compute each vertex degree $d(v_m)$ and each hyperedge degree $d(e_n)$

$$d\left(v_m\right) = \sum_{e\in\mathbf{E}} w\left(e\right) h\left(v_m, e\right) \tag{10}$$

$$d\left(e_n\right) = \sum_{v\in\mathbf{V}} h\left(v, e_n\right). \tag{11}$$

The normalized hypergraph Laplacian matrix L is computed using the incidence matrix H, hyperedge degrees, and vertex degrees. The following equation illustrates the calculation process:

$$L = Y - {D_v}^{-1/2} H W {D_e}^{-1} H^{\top} {D_v}^{-1/2} \tag{12}$$

where Y is the identity matrix and $D_v \in \mathbb{R}^{|M|\times|M|}$ and $D_e \in \mathbb{R}^{|N|\times|N|}$ represent the diagonal matrix of vertex degrees and hyperedge degrees, correspondingly.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:26:46 UTC from IEEE Xplore. Restrictions apply.

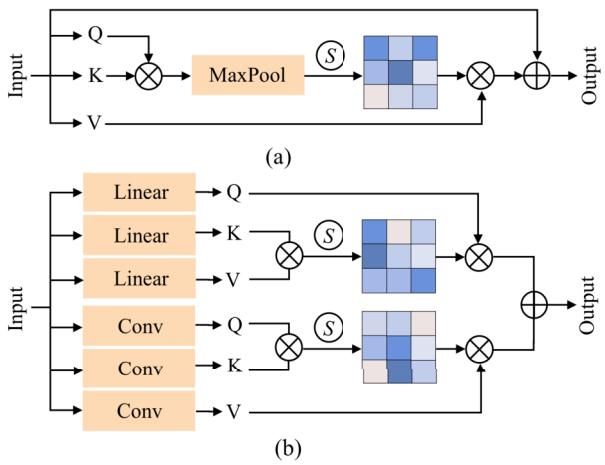


Fig. 4. Illustration of (a) vertex DAM and (b) FDAM.

The hypergraph structure can be constructed by the similarity or relationship between the vertices. In this method, the HSI can be easily encoded into the graph vertices through the encoder in (7), with a vertex corresponding to a superpixel in the HSI. For all given vertices, according to the spatial distance and spectral distance between vertices, we select the K-nearest vertices, forming the hyperedges E_{spa} and E_{spe}. Then, based on these hyperedges, we construct the spatial incidence matrix H_{spa} and spectral incidence matrix H_{spe}, obtaining the spatial space hypergraph G_{spa} and the spectral space hypergraph G_{spe}. To better represent the nonlinear higher order relationships in the HSI and reduce the computational complexity of the method, we fuse two hypergraphs to obtain the multiview hypergraph G_{multi}.

F. Differential Attention Module

The self-attention mechanism calculates the correlations among various elements in the input features, allowing the model to consider global information when processing each element and emphasizing more important information. This method is based on the self-attention mechanism and constructs two DAM for graph vertices, namely the VDAM and the FDAM.

1) Vertex DAM: Certain vertices may contain more significant information, while others may be background or noise. The vertex DAM employs an attention mechanism to adaptively learn important relationships between the vertices in the graph, helping the model better focus on the key regions in the HSI, rather than treating them equally as traditional graph-based methods do. As illustrated in Fig. 4(a), given the vertex feature map $\hat{\mathbf{S}} \in \mathbb{R}^{z\times b}$, the module first performs a tensor operation to reshape the feature dimensions for subsequent calculations. Then, we directly perform self-attention calculation on the input feature $\hat{\mathbf{S}}$, where the attention map undergoes a maximum pooling operation with a kernel size of 1. The VDAM is computed as

$$e_{ji}^{z} = \frac{\text{MaxPool}\left(\exp\left(\hat{\mathbf{S}}\cdot\hat{\mathbf{S}}^{\mathbf{T}}\right)\right)}{\sum_{i=1}^{N}\exp\left(\hat{\mathbf{S}}\cdot\hat{\mathbf{S}}^{\mathbf{T}}\right)} \tag{13}$$

where e_{ji}^{z} measures the influence of vertex i on vertex j and forms the vertex attention map $E^{b} \in \mathbb{R}^{z\times z}$ and MaxPool$(\cdot)$ represents the maximum pooling

$$\hat{\mathbf{S}}_{\text{out}} = \text{Attention}\left(\hat{\mathbf{S}}\right) + \hat{\mathbf{S}} = \mu\left(E^{b}\cdot\hat{\mathbf{S}}\right) + \hat{\mathbf{S}} \tag{14}$$

where μ is a learnable parameter.

2) Feature DAM: To better utilize the rich feature information of HSI, the FDAM captures the dependencies between different feature maps from both the global and local perspectives, allowing the model to weight based on the importance of different features and highlight important features while suppressing less relevant ones. As illustrated in Fig. 4(b), given a vertex feature map $\hat{\mathbf{S}} \in \mathbb{R}^{z\times b}$, the module also performs a tensor operation to reshape the feature dimensions for subsequent calculations. Then, for global differential attention, three initialized transformation matrices W_q, W_k, and W_v are multiplied with the features to obtain $\mathbf{Q}_g$, $\mathbf{K}_g$, and $\mathbf{V}_g$ for the attention calculation, where K is transposed to $\mathbb{R}^{1\times b\times z}$. Similarly, for local differential attention, we use three convolutional layers to perform a nonlinear transformation to obtain $\mathbf{Q}_l$, $\mathbf{K}_l$, and $\mathbf{V}_l$. Finally, the obtained global and local differential attention features are added together to get the final output $\hat{\mathbf{S}}_{\text{out}}$. The formula for calculating the FDAM can be written as follows:

$$\hat{\mathbf{S}}_{\text{out}} = \text{Softmax}\left(\frac{\mathbf{Q}_g\cdot\mathbf{K}_g^{\mathbf{T}}}{\alpha}\right)\mathbf{V}_g + \text{Softmax}\left(\frac{\mathbf{Q}_l\cdot\mathbf{K}_l^{\mathbf{T}}}{\beta}\right)\mathbf{V}_l \tag{15}$$

where α and β are the learnable scaling parameters.

G. Hypergraph Convolution

After constructing the hypergraph, we propagate features through HGC. Specifically, HGC involves two stages: 1) aggregating vertex features onto hyperedges to construct hyperedge features and 2) propagating aggregated hyperedge features back to the vertices, completing the vertex features update. Given the vertex features $\hat{\mathbf{V}}$, the propagation of information on the hypergraph can be computed as

$$\hat{\mathbf{V}}^{l+1} = \sigma\left(D_v^{-1/2}HWD_e^{-1}H^{\top}D_v^{-1/2}\hat{\mathbf{V}}^{l}\mathbf{\Theta}^{l}\right) \tag{16}$$

where $\mathbf{E}^{l} = H^{\top}D_v^{-1/2}\hat{\mathbf{V}}^{l}$ describes the process of aggregating vertex features and $\hat{\mathbf{V}}^{l+1} = D_v^{-1/2}HWD_e^{-1}\hat{\mathbf{E}}^{l}$ describes the process of aggregating hyperedge features. $\hat{\mathbf{V}}^{l}$ represents the vertex features of the lth layer, $\hat{\mathbf{V}}^{l+1}$ represents the updated vertex features, $\hat{\mathbf{E}}^{l}$ represents the aggregated hyperedge features of the lth layer, and W denotes the hyperedges weight matrix. For simplicity, W is set as an identity matrix in this method, $\mathbf{\Theta}^{l}$ represents the trainable parameters of HGC, and σ represents a nonlinear activation function.

H. Dynamic HGC

Most existing graph-based methods only use the initial graph structure, which neglects the fact that features undergo transformations after passing through hidden layers, and the corresponding structure of the transformed features may also change. To address this issue, this method employs a dynamic hypergraph CNN (DHGCN) [46] to adjust the hypergraph structure dynamically.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:26:46 UTC from IEEE Xplore. Restrictions apply.

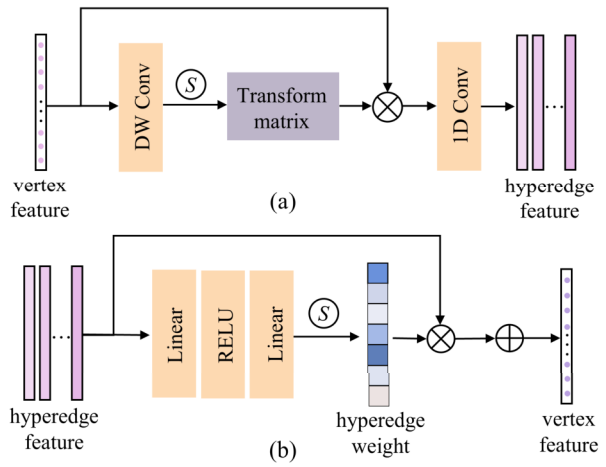


Fig. 5. Illustration of (a) vertex convolution transformation mechanism and (b) hyperedge convolution transformation mechanism.

DHGCN is also composed of hypergraph construction and HGC. This method also utilizes the KNN algorithm and mines the relationships between vertices by calculating the Euclidean distance and the cosine similarity between the features of each pair of vertices to construct the dynamic hypergraph. As a result, we obtain two hypergraphs G_{dis} and G_{sim}.

In static HGCN, the interaction between vertices-hyperedges-vertices in the HGC is essentially a simple summation of features. This may lead to an overestimation of the importance of irrelevant vertices. DHGCN applies an vertex convolution transformation mechanism (Vertex Conv) and an hyperedge convolution transformation mechanism (Hyperedge Conv) to transform the input features, dynamically extracting more complex higher order features, as shown in Fig. 5.

1) Vertex Convolution Transformation Mechanism: The neighbors of a vertex are analogous to the pixel neighborhoods in an image, which are typically considered a form of local structure. Depth-wise convolution can effectively capture such local spatial relationships. Therefore, we apply a depth-wise convolution operation to input vertex features to obtain a learnable transform matrix that can dynamically adjust the interaction weights between vertices. Given the input vertex features $\hat{\mathbf{V}}$

$$\mathrm{T} = \sigma\left(k^l \hat{\mathbf{V}}^{\mathbf{l}} + b^l\right) \tag{17}$$

where k^l denotes the trainable weight vector, b^l denotes the trainable bias, and T denotes the learnable transform matrix. Then, the transform matrix is multiplied by the vertex features to update the features of each vertex. Subsequently, a convolutional layer aggregates the updated neighbor features onto each hyperedge, resulting in the features of each hyperedge

$$\hat{\mathbf{E}}^{\mathbf{l}} = conv\left(\mathbf{T} \cdot \hat{\mathbf{V}}^{\mathbf{l}}\right) \tag{18}$$

where $\hat{\mathbf{E}}^{\mathbf{l}}$ represents the aggregated hyperedge features.

2) Hyperedge Convolution Transformation Mechanismfusing multispectral images and: The structure and scale of hyperedges vary significantly, and the correlation between hyperedges may depend on the information from the entire graph. MLP can directly model global dependencies through its fully connected structure, while the convolution operation is more focused on local relationships. We use an MLP to assign a weight to each hyperedge connected to a vertex v_i

$$\mathbf{F} = \mathrm{MLP}\left(\mathbf{E}_{v_i}\right) \tag{19}$$

where $\mathbf{E}_{v_i}$ denotes all hyperedges connected with the vertex v_i, $\mathbf{F} = \{f_1, f_2, \ldots, f_i, \ldots, f_{k_{v_i}}\}$ is the weight of all hyperedges connected with the vertex v_i, and k_{v_i} represents the number of all hyperedges connected with the vertex v_i. Then, based on the computed weights, the features of each hyperedge are aggregated onto the vertex, and the updated vertex features $\hat{v}_i$ are obtained

$$\hat{v}_i = \sum_{n=1}^{k_{v_i}} f_n e_n. \tag{20}$$

Since we have constructed two hypergraphs, after two transformation mechanisms, we need to fuse the two updated vertex feature representations corresponding to different types of hyperedges to generate a new updated vertex feature representation. Finally, after passing through a dropout layer and an MLP enhancement model to enhance the generalization ability, the final updated vertex features are obtained.

TABLE I
LAND COVER CATEGORIES AND DATASET DIVISION FOR THE PU DATASET

No	Color	Class	Train	Validation	Test
1		Asphalt	10	1	6620
2		Meadows	10	1	18638
3		Gravel	10	1	2088
4		Trees	10	1	3053
5		Metal Sheets	10	1	1334
6		Bare Soil	10	1	5018
7		Bitumen	10	1	1310
8		Bricks	10	1	3671
9		Shadows	10	1	936
		Total	90	9	42677

III. EXPERIMENTS

To fully assess the classification performance of the MDHGC, we employed four HSI datasets.

A. Datasets

1) University of Pavia: Fig. 6(a) and (b) present the University of Pavia (PU) dataset that has 103 spectral channels (430–860 nm), with a spatial resolution of 1.3 m, and an image dimension of 610 × 340 pixels. The dataset contains 42 776 labeled pixels, categorized into nine distinct classes. The details of each class and dataset partitioning are provided in Table I.

2) Indina Pines: Fig. 7(a) and (b) present the Indian Pine (IP) dataset that encompasses 200 spectral channels (400–2500 nm), with a spatial resolution of 20 m and an image dimension of 145 × 145 pixels. Among them, 10 249 labeled pixels are classified into 16 categories. The specific category definitions and dataset division details are provided in Table II.

3) Salinas: Fig. 8(a) and (b) presents the Salinas (SA) dataset that contains 204 spectral channels (400–2500 nm), with a spatial resolution of 3.7 m and an image dimension of 512 × 217 pixels. Among them, 54 129 labeled pixels are classified into 16 categories for research and analysis. The specific category definitions and dataset division are provided in Table III.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:26:46 UTC from IEEE Xplore. Restrictions apply.

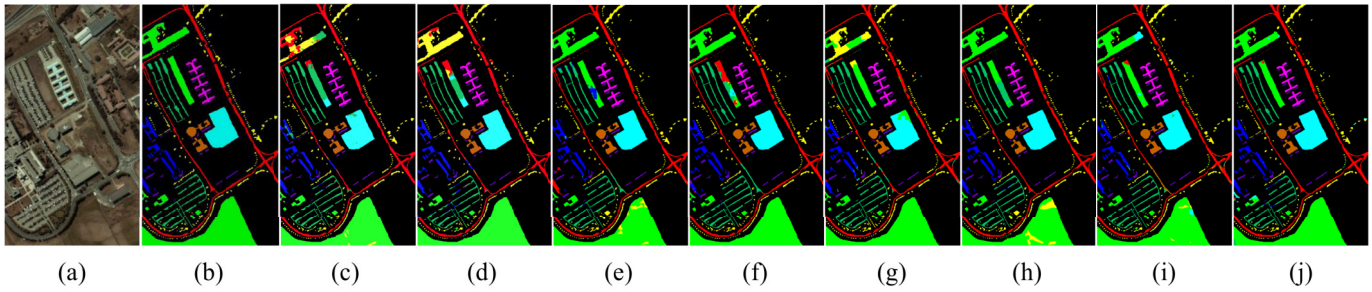


Fig. 6. Qualitative results of various models on the PU dataset. (a) False-color image. (b) Ground truth map. (c) 2D-CNN. (d) GCN. (e) CEGCN. (f) WFCG. (g) AMGCFN. (h) HGCN-MHF. (i) DF2Net. (j) MDHGC.

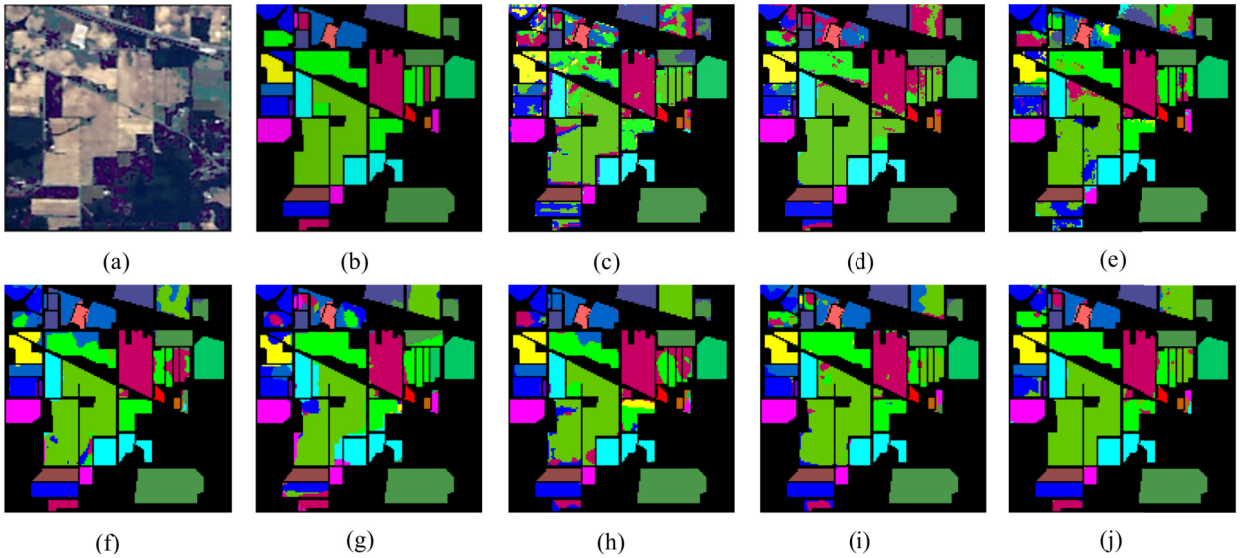


Fig. 7. Qualitative results of various models on the IP dataset. (a) False-color image. (b) Ground truth map. (c) 2D-CNN. (d) GCN. (e) CEGCN. (f) WFCG. (g) AMGCFN. (h) HGCN-MHF. (i) DF2Net. (j) MDHGC.

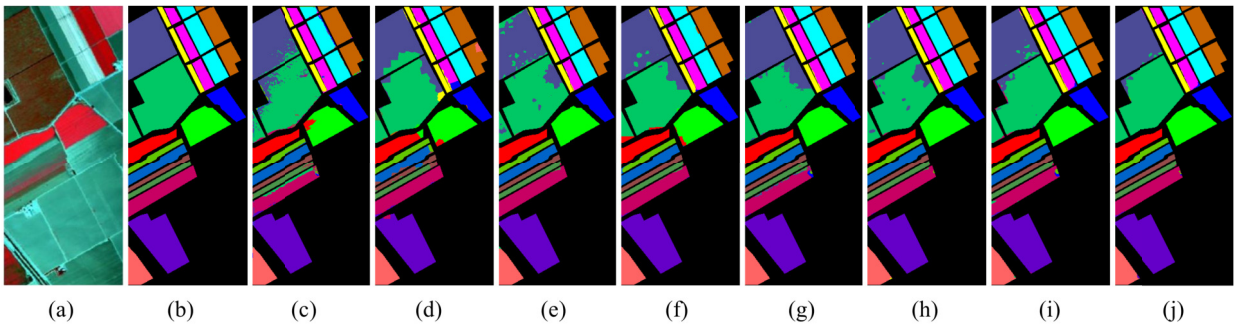


Fig. 8. Qualitative results of various models on the SA dataset. (a) False-color image. (b) Ground truth map. (c) 2D-CNN. (d) GCN. (e) CEGCN. (f) WFCG. (g) AMGCFN. (h) HGCN-MHF. (i) DF2Net. (j) MDHGC.

4) Liao Ning-01: The Liao Ning-01 (LN) dataset is a dataset obtained by fusing multispectral images and hyperspectral images acquired by the Zhongxing-1 02D satellite in the northwest of Jinpu New District, Dalian City, Liaoning Province, using the fusion method of unsupervised deep spectral–spatial collaborative constraint [18]. The size of the fused data is 900 × 900 × 166, which is shown in Fig. 9(a). There are a total of 265 004 pixels with ground truth labels, and it is divided into ten different categories. The details of each class and dataset partitioning are provided in Table IV. It should be highlighted that, due to the large size of the LN dataset, we averagedly divided it into two parts for classification.

B. Experiment Settings

This section presents the detailed parameter configurations and the evaluation metrics for the experiments of the MDHGC. Then, we provide an overview of all advanced models used for comparison, along with their parameter settings.

1) Configuration Details: The model runs for 400 epochs, trained using the Adam optimizer. The initial learning rate for all datasets is 0.001, with a weight decay coefficient of 0.0005.

To conduct a comprehensive evaluation of the model's performance and benchmark it against advanced models, the following evaluation metrics are used for classification performance: per-class accuracy (PA), overall accuracy (OA), average accuracy (AA), and kappa coefficient (Kappa). Con-

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:26:46 UTC from IEEE Xplore. Restrictions apply.

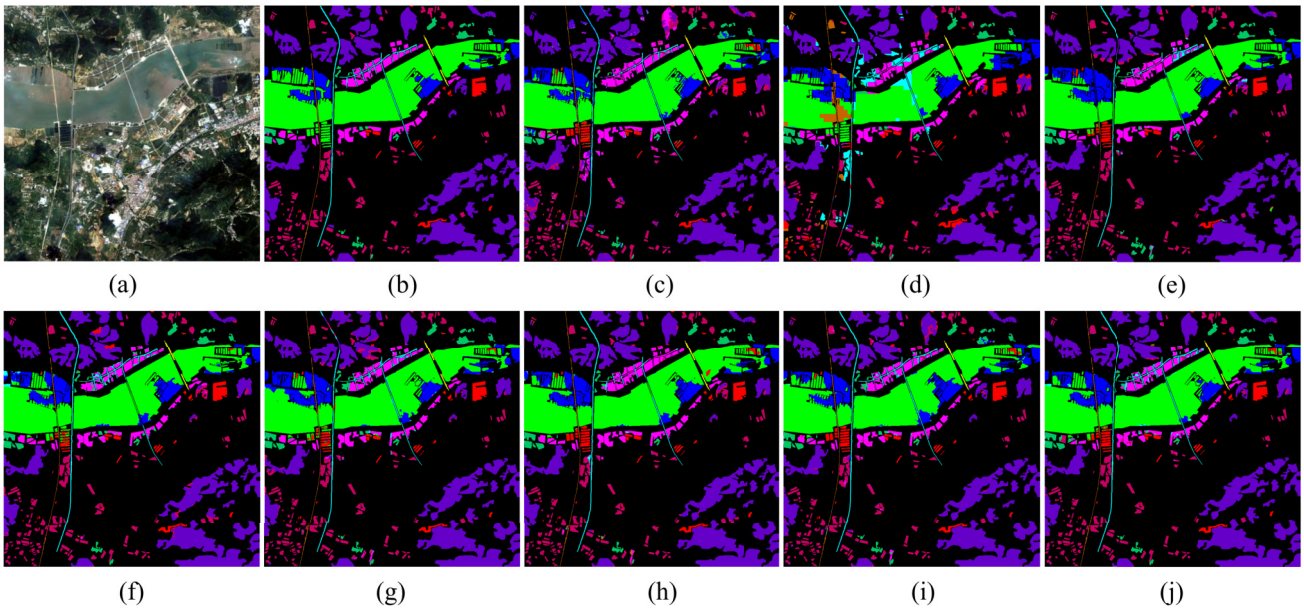


Fig. 9. Qualitative results of various models on the LN dataset. (a) Fusion result. (b) Ground truth map. (c) 2D-CNN. (d) GCN. (e) CEGCN. (f) WFCG. (g) AMGCFN. (h) HGCN-MHF. (i) DF2Net. (j) MDHGC.

TABLE II
LAND COVER CATEGORIES AND DATASET DIVISION FOR THE IP DATASET

No	Color	Class	Train	Validation	Test
1		Alfalfa	10	1	35
2		Corn-notill	10	1	1417
3		Corn-mintill	10	1	889
4		Corn	10	1	226
5		Grass-pasture	10	1	472
6		Grass-tress	10	1	719
7		Grass-pasture-mowed	10	1	17
8		Hay-windrowed	10	1	467
9		Oats	10	1	9
10		Soybean-notill	10	1	961
11		Soybean-mintill	10	1	2444
12		Soybean-clearn	10	1	582
13		Wheats	10	1	194
14		Woods	10	1	1254
15		Bldg-Grass-Trees-Drivers	10	1	375
16		Stone-Stell-Towers	10	1	82
		Total	160	16	10073

TABLE III
LAND COVER CATEGORIES AND DATASET DIVISION FOR THE SA DATASET

No	Color	Name	Train	Validation	Test
1		Broccoli-green-weeds-1	10	1	2008
2		Broccoli-green-weeds-22	10	1	3725
3		Fallow	10	1	1975
4		Fallow-rough-plow	10	1	1393
5		Fallow-smooth	10	1	2677
6		Stubble	10	1	3958
7		Celery	10	1	3578
8		Grapes-untrained	10	1	11270
9		Soil-vineyard-develop	10	1	6202
10		Corn-weeds	10	1	3277
11		Lettuce-romaine-4wk	10	1	1067
12		Lettuce-romaine-5wk	10	1	1926
13		Lettuce-romaine-6wk	10	1	915
14		Lettuce-romaine-7wk	10	1	1069
15		Vineyard-untrained	10	1	7267
16		Vineyard-vertical-trellis	10	1	1806
		Total	160	16	53953

cretely, PA evaluates the accuracy for each class, providing a detailed view of the model's performance across different categories. OA calculates the total percentage of correctly classified pixels across all classes, offering a global performance measure. AA computes the mean of the per-class accuracies, providing an average measure of the classification performance across all classes. The Kappa coefficient assesses the consistency between true labels and predicted labels, adjusting for the possibility of agreement occurring by chance.

TABLE IV
LAND COVER CATEGORIES AND DATASET DIVISION FOR THE LN DATASET

No	Color	Name	Train	Validation	Test
1		Reservoir	20	2	5454
2		Seawater	20	2	100254
3		Sandy soil	20	2	14565
4		Broken bridge	20	2	512
5		Barren grass	20	2	16477
6		Bare soil	20	2	5561
7		Highway	20	2	1408
8		Railway	20	2	6265
9		Mountain vegetation	20	2	95049
10		Arable land	20	2	19239
		Total	200	20	264784

To mitigate the influence of random factors, all experimental results are independently carried out ten times, and the final outcomes are based on the average values. The experiments were conducted using Python 3.8.20 and PyTorch 1.10.0, in an environment with an i7-10700K CPU, 32 GB of RAM, and an NVIDIA GeForce RTX 3090 GPU.

2) Comparison Methods: In order to assess the classification performance of the MDHGC, it is compared with seven distinct classification models, including two traditional methods, i.e., 2D-CNN [21] and GCN [47]; three hybrid methods combining CNN and GCN, i.e., CEGCN [33], WFCG [34], and AMGCFN [35]; and two hypergraph-based methods, i.e., DF2Net [41] and HGCN-MHF [40]. All experiments are conducted using the hyperparameters recommended in the original articles of these methods. To ensure fairness, the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:26:46 UTC from IEEE Xplore. Restrictions apply.

TABLE V
QUANTITATIVE RESULTS OF VARIOUS MODELS ON THE PU DATASET

Class	2D-CNN	GCN	CEGCN	WFCG	AMGCFN	HGCN-MHF	DF2Net	MDHGC
1	58.21±6.11	44.49±2.08	97.26±1.89	96.84±1.21	89.75±5.85	**98.90±0.66**	93.65±5.01	92.27±3.13
2	95.59±1.69	85.64±6.66	92.31±3.42	89.57±6.24	86.93±6.09	91.45±3.91	97.00±0.63	**99.43±0.49**
3	74.47±7.16	80.90±3.18	91.28±7.37	**98.43±1.77**	93.46±6.24	93.10±6.27	87.83±3.03	92.22±8.99
4	92.63±1.20	63.14±1.71	93.24±1.78	94.59±1.70	88.42±3.79	94.54±2.47	93.17±1.35	**95.01±0.95**
5	98.60±0.66	99.96±0.09	99.99±0.03	99.90±0.21	99.84±0.18	**100.00±0.00**	99.94±0.09	**100.00±0.00**
6	98.67±0.32	98.67±1.04	98.83±0.49	99.42±0.60	97.43±2.12	99.96±0.08	98.32±2.73	**100.00±0.00**
7	93.92±1.10	95.48±3.40	99.79±0.26	**99.99±0.03**	99.83±0.33	98.28±1.27	98.74±1.98	99.80±0.21
8	57.86±4.94	94.50±0.46	98.80±1.37	97.86±2.27	93.96±5.86	**99.29±0.53**	91.90±5.88	97.17±1.57
9	99.40±0.26	26.77±3.52	97.76±2.91	98.69±2.74	89.55±8.55	98.55±1.90	99.91±0.08	**99.96±0.06**
OA (%)	85.79±1.33	79.17±3.08	93.97±1.88	94.20±2.59	90.45±2.47	95.26±1.98	95.69±1.35	**97.57±0.64**
AA (%)	85.48±1.08	76.62±1.53	96.58±0.97	93.60±2.36	93.10±1.60	97.29 ±1.01	94.15±2.04	**97.32±0.97**
Kappa ×100	81.48±1.68	73.34±3.60	93.48±2.43	92.49±3.26	87.68±3.04	93.84±2.54	94.31±1.77	**96.78±0.85**

TABLE VI
QUANTITATIVE RESULTS OF VARIOUS MODELS ON THE IP DATASET

Class	2D-CNN	GCN	CEGCN	WFCG	AMGCFN	HGCN-MHF	DF2Net	MDHGC
1	91.67±4.21	97.22±1.11	97.22±3.51	99.00±2.24	98.06±2.79	98.88±1.37	97.78±1.11	**100.00±0.00**
2	58.62±5.29	75.46±10.95	61.23±8.04	76.76±6.91	67.84±9.40	77.12±7.98	78.32±2.43	**88.08±6.26**
3	46.95±5.34	72.41±8.46	73.58±14.07	**83.43±7.19**	82.60±4.60	80.54±10.65	78.20±5.04	83.41±1.96
4	87.84±4.45	81.94±6.12	89.43±7.67	98.58±2.27	93.87±4.24	97.35±4.57	99.12±0.92	**99.12±1.15**
5	83.09±2.51	82.41±2.49	**89.07±2.49**	87.66±4.43	88.20±5.71	87.95±4.78	88.88±1.83	88.58±0.70
6	91.72±1.64	84.67±4.58	**99.44±0.47**	97.93±0.96	92.92±4.30	98.46±0.98	99.08±1.17	99.17±0.67
7	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**
8	99.70±0.50	99.57±1.44	98.93±1.16	99.73±0.59	99.79±0.29	99.94±0.14	99.91±0.17	**100.00±0.00**
9	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**
10	74.35±2.91	73.51±5.04	76.96±9.13	**87.14±3.92**	82.07±7.41	80.17±5.66	81.58±3.39	85.65±2.64
11	69.87±2.50	82.72±1.11	71.70±14.74	81.39±6.01	78.49±7.88	86.82±8.19	87.93±3.96	**91.37±2.04**
12	52.28±4.18	87.82±2.90	71.78±19.90	82.68±11.19	77.57±12.97	**90.99±7.04**	87.92±4.42	90.39±2.12
13	**100.00±0.00**	97.13±1.80	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**
14	87.39±2.71	80.92±4.05	94.08±4.29	95.66±4.91	96.07±2.13	95.75±3.85	98.85±0.56	**99.60±0.06**
15	94.10±1.66	98.40±1.13	84.01±14.65	89.33±8.74	96.75±2.95	94.30±7.55	**100.00±0.00**	100.00±0.21
16	99.76±0.48	98.80±1.57	99.76±0.48	99.70±0.67	97.23±2.53	99.76±0.72	**100.00±0.00**	98.80±0.95
OA (%)	73.88±1.12	82.04±1.31	79.53±2.39	86.80±2.71	83.96±2.48	88.11±2.51	89.04±0.87	**92.29±0.84**
AA (%)	83.58±0.86	88.31±0.47	87.95±2.30	86.19±1.19	90.60±1.17	93.00±1.17	93.60±0.55	**95.26±0.46**
Kappa ×100	70.38±1.28	79.61±1.45	76.78±2.69	85.01±3.03	81.87±2.75	86.50±2.76	87.51±0.97	**91.20±0.96**

selected comparative methods using GCN all employed the SLIC algorithm.

C. Classification Results

Tables V–VIII present the quantitative evaluation results of various models across four datasets, with the highest values bolded. Additionally, the qualitative evaluation results are presented in Figs. 6–9. The results demonstrate that MDHGC outperforms all other comparative algorithms on all datasets.

1) University of Pavia: The PU dataset features complex land cover distributions, making classification more challenging. According to the results in Table V, the accuracy of the GCN algorithm is inferior to that of 2D-CNN, which is attributed to the presence of many fine-grained objects in the PU dataset. GCN performs worse than CNN in classifying fine-grained objects. For instance, as shown in Fig. 6, GCN exhibits poor performance for "Shadow" and "Asphalt," leading to a significant number of misclassifications in complex environments. The hybrid networks CEGCN, WFCG, and AMGCFN improve the OA values by approximately 8%, 8%, and 4%, respectively, compared to 2D-CNN. This is because hybrid networks extract complementary features at different levels and better utilize the topological structure information of HSI, reducing the generalization error of a single model. Among them, WFCG introduces a graph attention network and a weighted feature fusion, which more effectively capture features and integrate topological structure information, achieving an OA value of 94.20%. However, all three methods are based on simple graph improvements, and they suffer from insufficient expressive power when handling complex higher order information. The hypergraph-based methods HGCN-MHF and DF2Net outperform the simple graph methods, with OA values improving by at least 0.41%. As shown in Fig. 6, the hypergraph networks demonstrate superior classification performance, especially for "Shadow" and "Asphalt" in complex environments. Among all methods, our MDHGC achieves the best classification performance. For instance, MDHGC improves the classification accuracy of "Meadows" by approximately 2.5%, and both the OA and Kappa values increase by about 2%, with the method showing the most stable performance across multiple experiments.

2) Indina Pines: The IP dataset has relatively low spatial resolution and an extremely imbalanced sample distribution,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:26:46 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
QUANTITATIVE RESULTS OF VARIOUS MODELS ON THE SA DATASET

Class	2D-CNN	GCN	CEGCN	WFCG	AMGCFN	HGCN-MHF	DF2Net	MDHGC
1	98.82±2.36	95.28±2.36	99.99±0.02	99.99±0.02	96.92±3.94	**100.00±0.00**	99.98±0.04	99.97±0.13
2	95.72±1.46	97.87±1.74	**100.00±0.00**	**100.00±0.00**	99.56±1.04	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**
3	99.67±0.16	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	99.95±0.10	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**
4	**100.00±0.00**	98.37±3.27	99.86±0.22	99.84±0.33	98.12±2.78	**100.00±0.00**	99.60±0.20	99.85±0.15
5	85.03±2.68	94.87±2.68	97.95±1.26	97.88±1.10	96.34±3.26	**97.97±1.30**	97.76±1.02	97.02±0.86
6	97.85±1.76	97.40±2.43	**99.96±0.05**	99.59±0.40	99.59±0.57	99.86±0.07	99.82±0.12	99.90±0.14
7	**100.00±0.00**	**100.00±0.00**	99.97±0.04	99.94±0.08	99.68±0.36	99.69±0.46	99.97±0.06	**100.00±0.00**
8	87.06±0.58	87.58±2.33	85.72±4.05	87.76±2.87	88.29±3.87	89.32±3.64	93.71±0.59	**98.27±0.96**
9	98.76±1.64	98.76±1.64	**100.00±0.00**	**100.00±0.00**	99.99±0.02	99.98±0.04	99.85±0.08	**100.00±0.00**
10	96.86±1.92	96.86±1.92	94.65±3.41	**97.72±1.00**	93.46±5.34	91.78±9.30	95.81±0.74	97.05±0.62
11	**100.00±0.00**	**100.00±0.00**	99.28±0.37	99.95±0.12	98.77±1.65	99.87±0.26	99.87±0.14	99.81±0.06
12	88.42±2.01	87.90±2.01	**100.00±0.00**	99.99±0.03	99.95±0.10	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**
13	78.87±5.19	90.73±5.19	**99.96±0.05**	99.83±0.21	99.54±0.68	99.65±0.39	99.89±0.14	98.88±0.32
14	86.70±1.73	86.70±1.73	99.60±0.34	97.22±2.50	99.59±0.34	99.15±1.42	98.11±0.40	**99.62±0.13**
15	84.83±0.36	94.36±0.97	94.54±4.01	90.75±5.06	92.20±8.77	97.72±1.90	96.24±0.80	**98.04±0.49**
16	**100.00±0.00**	**100.00±0.00**	99.54±0.57	99.81±0.22	99.87±0.17	99.80±0.25	99.60±0.52	**100.00±0.02**
OA (%)	92.67±0.33	94.79±0.69	95.81±0.41	95.84±0.75	95.64±1.31	96.86±0.96	97.72±0.22	**99.01±0.20**
AA (%)	93.66±0.48	95.55±0.43	98.20±0.21	98.14±0.32	97.61±0.77	98.42±0.70	98.76±0.14	**99.28±0.08**
Kappa ×100	91.84±0.37	94.21±0.76	95.35±0.46	95.38±0.83	95.15±1.46	96.44±1.06	97.46±0.25	**98.89±0.22**

TABLE VIII
QUANTITATIVE RESULTS OF VARIOUS MODELS ON THE LN DATASET

Class	2D-CNN	GCN	CEGCN	WFCG	AMGCFN	HGCN-MHF	DF2Net	MDHGC
1	97.04±2.04	94.82±3.19	97.65±2.70	98.79±0.47	95.79±3.41	99.36±1.41	99.56±0.50	**99.72±0.32**
2	90.86±0.84	82.98±3.81	91.96±1.99	89.57±1.49	89.33±4.71	89.90±2.68	90.85±1.49	**94.53±0.60**
3	94.71±2.52	92.29±3.24	89.97±3.86	96.33±2.01	91.84±6.77	96.50±3.48	95.20±2.01	**97.36±1.28**
4	97.33±1.20	96.63±2.91	99.24±0.45	98.36±1.16	91.49±2.19	**99.73±1.86**	97.21±1.16	99.41±0.28
5	91.22±2.31	89.83±5.63	91.10±1.89	93.94±0.58	89.27±2.92	92.44±3.52	88.94±0.58	**95.26±1.68**
6	73.19±7.24	65.85±11.58	59.69±5.78	90.17±2.59	72.91±7.08	88.45±5.40	92.36±2.59	**95.83±1.34**
7	95.43±2.20	64.45±15.52	80.93±8.24	79.72±8.88	88.76±13.58	95.54±7.39	96.36±8.88	**98.34±0.67**
8	82.59±7.59	91.87±6.62	85.12±2.08	93.68±3.74	90.84±5.24	**95.71±5.75**	91.56±3.74	94.58±1.01
9	90.86±1.64	89.72±1.34	96.42±1.21	95.90±2.73	95.77±1.98	**98.28±1.28**	97.50±2.73	94.86±0.38
10	68.76±6.61	67.41±15.38	**94.60±2.40**	94.52±1.84	89.96±1.95	93.47±1.01	88.28±1.84	90.80±1.33
OA(%)	89.12±1.18	85.20±3.88	92.74±1.40	93.11±1.12	91.65±2.52	93.94±1.54	93.37±1.12	**94.74±0.35**
AA(%)	88.20±1.93	83.58±6.51	88.67±1.53	93.10±1.32	89.62±3.97	94.94±1.30	93.78±1.32	**95.49±0.51**
Kappa×100	85.14±1.32	80.17±4.54	90.02±1.88	90.56±1.49	85.29±3.29	91.69±2.06	90.89±1.49	**92.75±0.50**

which leads to lower classification accuracy compared to other datasets. Specifically, according to the results in Table VI, the GCN algorithm performs better than 2D-CNN in terms of classification accuracy, with 2D-CNN also showing considerable "salt-to-pepper" noise in Fig. 7. The low spatial resolution of the IP dataset leads to pixel confusion, and while CNN is strong at extracting local features, it is sensitive to noise, which reduces classification accuracy. In contrast, GCN, by leveraging the graph structure, is better able to combat noise interference. For hybrid networks, CEGCN concatenates features, which increases the final feature dimension, resulting in insufficient information fusion and the presence of redundant information, thus leading to suboptimal classification performance. This also explains why there is a significant gap between the OA and AA values of CEGCN. WFCG and AMGCFN employ weighted fusion and cross-attention fusion strategies, achieving OA values of 86.8% and 83.96%, respectively, on the IP dataset. HGCN-MHF and DF2Net, which utilize hypergraph structures, achieve even higher OA values of 88.11% and 89.04%, respectively. In comparison, MDHGC, which combines attention mechanisms and dynamic hypergraph structures, fully exploits the spatial–spectral information and deep semantic features of HSI, achieving the best results in both qualitative and quantitative evaluations.

3) Salinas: The SA dataset has a very balanced sample distribution, with smooth and regular land cover distribution, which leads to good classification results across all algorithms. As shown in Table VII, the OA value for the GCN algorithm is about 2% higher than that of 2D-CNN. This is due to the graph structure's strong ability to represent nonlocal features, with GCN performing better in smooth areas compared to CNN. The classification results of all hybrid networks are superior to those of individual networks, further validating that hybrid networks have more substantial representation capabilities. According to the results in Fig. 8, for "Grapes-untrained" and "Vineyard-untrained," most deep learning-based comparison algorithms perform poorly. This is because these two classes have highly similar spectral features and high intraclass variability. Hypergraphs model higher order relationships between multiple samples through hyperedges, indirectly reinforcing intraclass consistency and reducing cross-class confusion. Hypergraph-based algorithms

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:26:46 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
QUANTITATIVE RESULTS OF THE ABLATION OF EACH MODULE ON FOUR DATASETS

MSFE	GLFM	HGCN	DHGCN	DAM	University of Pavia		Indian Pine		Salinas		Liao Ning-01	
					OA(%)	Kappa×100	OA(%)	Kappa×100	OA(%)	Kappa×100	OA(%)	Kappa×100
-	-	-	-	-	94.56±1.81	92.91±2.30	86.06±1.30	84.14±1.47	97.03±0.89	96.69±0.98	91.75±1.29	89.70±1.67
✓	-	-	-	-	94.99±1.81	93.45±1.38	87.86±1.53	86.20±1.72	97.45±0.73	97.15±0.55	92.38±1.09	90.34±1.45
✓	✓	-	-	-	95.33±1.37	93.83±1.58	88.14±1.49	86.43±1.59	97.77±0.56	97.48±0.54	93.13±0.67	91.61±0.90
✓	✓	✓	-	-	96.37±1.07	95.23±1.38	90.29±0.86	88.90±0.98	98.40±0.32	98.28±0.20	94.01±0.71	91.72±0.87
✓	✓	✓	✓	-	96.92±0.94	95.70±1.16	91.38±0.98	90.13±1.11	98.89±0.15	98.77±0.17	94.44±0.23	92.31±0.31
✓	✓	✓	✓	✓	**97.57±0.64**	**96.78±0.85**	**92.29±0.84**	**91.20±0.96**	**99.01±0.20**	**98.89±0.22**	**94.74±0.35**	**92.78±0.50**

performed well on these two classes, achieving at least a 1% and 2% improvement in accuracy compared to simple graph algorithms. MDHGC achieved the optimal classification performance, with an average OA value of 99.01%, and showed the most stable results across multiple experiments. The classification map of MDHGC also demonstrates the smoothest predictions, most consistent with the ground truth reference map.

4) Liao Ning-01: The LN dataset consists of numerous samples with a complex distribution of features, which greatly increases the difficulty of feature extraction and classification tasks. According to Table VIII, the accuracy of 2D-CNN exceeds that of the GCN algorithm by about 4%. This is because there are many fine features in this dataset, and CNN has a higher ability to extract fine features than GCN. For example, from Fig. 9, it is noted that the GCN performs poorly on "Highway" and "Seawater," and there are many misclassification phenomena in complex environments. The hybrid network can extract features at different levels and fully utilize the advantages of different network structures. The OA value has been significantly improved compared to the single network structure. From Fig. 9, it can be seen that there are some "salt-to-pepper" phenomena in the classification results of 2D-CNN, and the misclassification phenomena of the classification results of GCN are widespread. The hybrid networks CEGCN, WFCG, and AMGCFN have solved these problems well. However, these three hybrid networks are all based on the improvement of simple graphs, and there will be problems of insufficient expression ability when dealing with complex relationships and high-order information. For example, in the classification of the Highway category, most algorithms based on simple graphs cannot distinguish Highways on the sea surface. While the methods based on hypergraphs can model the complex high-order interaction relationships between vertices and show better classification performance for "Highways" and "Reservoir." Among them, our proposed MDHGC attains the top OA and Kappa values and has performed the most stably in multiple experiments.

IV. DISCUSSION AND ANALYSIS

A. Ablation Study

To evaluate the performance of each module, we carried out multiple ablation experiments. Table IX presents the average OA and Kappa values of the four datasets under different module combinations. The baseline model refers to the pixel-level branch of MDHGC that has removed the MSFE and the self-attention mechanisms of GLFM. Through comprehensive experiments on four datasets, it has been validated that the model's constituent modules each play a significant role in achieving performance enhancement. Specifically, from the MSFE, it is evident that aggregating multiscale information is crucial for feature extraction. The GLFM verified the effectiveness of fusing local and global information for learning features. The HGCN module is capable of mining high-order interaction relationships, and the dual-branch hybrid model enables a more comprehensive analysis of the data. After joining DHGCN, the model can dynamically adjust the network structure, better adapt to complex feature interactions, and obtain more accurate classification results. By comparing the inclusion of DAM, the OA and Kappa values of all four datasets show improvement. This enhancement is because the HGCN focuses better on the key graph regions and key vertex features.

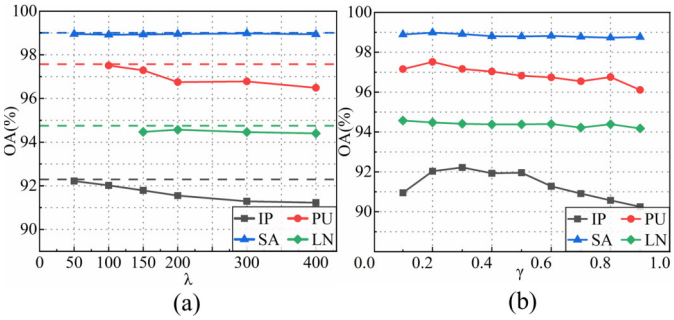


Fig. 10. OA of MDHGC with different parameters on each dataset. (a) Different superpixel segmentation methods. (b) Different fusion coefficients γ.

B. Superpixel Segmentation Analysis

To validate the effectiveness of the proposed A2SLIC method, we conducted a comparative analysis between A2SLIC and the traditional SLIC method. Specifically, the segmentation scales of the traditional SLIC algorithm were set to 50, 100, 150, 200, 300, and 400 to ensure a fair comparison. Experiments were then conducted on four datasets, with each experiment independently repeated ten times, and the average results reported. The results are shown in Fig. 10(a), where the dotted line represents the classification results of MDHGC based on the A2SLIC method, and the solid line represents the classification results of MDHGC based on the traditional SLIC method. Due to memory limitations, the PU dataset did not show relevant accuracy at $\lambda = 50$, and no relevant accuracy was observed for the LN dataset at $\lambda < 150$. It can be observed

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:26:46 UTC from IEEE Xplore. Restrictions apply.

TABLE X
OA (%) OF DIFFERENT FUSION STRATEGIES ON FOUR DATASETS

Fusion Strategy	PU	IP	SA	LN
Concatenate	96.26±0.94	90.12±1.13	98.42±0.32	94.27±0.21
Cross-Attention [35]	96.75±1.25	91.16±1.00	98.62±0.20	94.19±0.29
Weighted-Add	**97.57±0.64**	**92.29±0.84**	**99.01±0.20**	**94.74±0.35**

that the MDHGC results based on A2SLIC outperform all results derived from the traditional SLIC method.

It is worth noting that, although A2SLIC can adaptively adjust the number of superpixels for different datasets, avoiding the need for manual adjustment of segmentation scales, SLIC-based methods still tend to generate superpixels with relatively uniform sizes and shapes. When processing images with significant content variation, this may lead to both over-segmentation and under-segmentation. To address this issue, other superpixel segmentation methods, such as ERS [48], have also been applied in the hyperspectral domain. However, superpixel segmentation methods typically satisfy only a subset of evaluation metrics, with improvements in one attribute potentially leading to deterioration in another [49]. For instance, the superpixels generated by ERS exhibit lower compactness and smoothness, and the algorithm has high computational complexity. Therefore, the optimal choice should be a tradeoff based on the core requirements of the specific application scenario and the available resources [50]. Furthermore, current superpixel segmentation methods are primarily designed for RGB images. When these methods are applied in hyperspectral domains, the spectral dimensions of the HSI are often reduced, which may have varying effects on segmentation quality [51]. However, there is currently no superpixel segmentation method specifically designed for HSI, making superpixel segmentation in the hyperspectral domain an ongoing and challenging task.

C. Fusion Strategy Analysis

Feature fusion across different levels is a key technique for enhancing model performance, with the core goal of maximizing complementary information while minimizing redundant noise. Previous research has demonstrated the effectiveness of combining features from different levels for HSIC. There are many ways to fuse features, and choosing the appropriate fusion method is critical for model performance, efficiency, and task adaptability. In order to prove the effectiveness of the weighted fusion approach, a comparison is made between the model with various feature fusion methods. Table X presents the OA values for each variant, with the highest values highlighted in bold. "Weighted-Add" refers to the complete MDHGC, "Concatenate" refers to fusing the two branches of MDHGC by concatenating their features, and "Cross Attention [35]" refers to fusing the two branches of MDHGC through cross-attention. By observing the results from the four datasets, MDHGC consistently performs well under different fusion methods, which demonstrates that feature fusion across different levels improves the model's capacity to detect complex patterns; and it is also evident that the MDHGC with weighted fusion achieves the best classification performance.

To determine the optimal fusion coefficient γ, this section sets the fusion coefficient values to 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, and 0.9 across four datasets to analyze how the fusion coefficient affects the model's performance. Fig. 10(b) presents the average values of the OA metric. The results from the four datasets show a similar trend. When γ is 0.2, the PU and SA datasets achieve the best OA. When γ is 0.3 or 0.1, the IP dataset and the LN dataset achieve the best OA, respectively. Therefore, in all the experiments, the fusion coefficient γ for the PU, IP, SA, and LN datasets is set to 0.2, 0.3, 0.2, and 0.1, respectively.

D. Generalization Performance

To compare the model's generalization ability with randomly sampled data at different quantities, we tested MDHGC and other comparison models on four datasets under different training labeled sample sizes. Specifically, we randomly selected 5, 10, 15, 20, 25, and 30 labeled samples per class as training samples each time, with one sample per class selected as the validation sample. The average values of the OA metric are shown in Fig. 11. It can be seen that the performance of each algorithm improves as the training sample size increases. As the training sample size decreases, the performance of each algorithm drops to varying degrees. The performance of MDHGC on the PU and SA datasets is slightly lower than that of HGCN-MHF when randomly selecting five samples per class. However, the performance consistently outperforms HGCN-MHF with the increase in training samples. This may be due to the insufficient dynamic relationship modeling in the dynamic hypergraph of MDHGC under small sample sizes, which can lead to overfitting of noise and poor generalization. When the sample size is sufficiently large, MDHGC is able to capture more detailed feature patterns, enhancing its ability to distinguish complex land cover types. Since the LN dataset was divided into two parts for training, the total sample size for each category was slightly larger than that of the other three datasets. Therefore, MDHGC consistently performed the best on the LN dataset.

E. Model Computational Complexity

To evaluate the computational complexity and efficiency of MDHGC, we use network parameters (Par), giga floating-point (GFP) operations per second, and training time [T(s)] as performance metrics. Experiments were conducted on the PU, IP, SA, and LN datasets, and the results are presented in Table XI. Among them, "Base" refers to the model established by using only the CNN branch of MDHGC and two depthwise separable convolution operations. It is evident that MDHGC has a relatively high computational complexity. This is because the dynamic hypergraph requires the reconstruction of the hypergraph structure each time to adjust the graph structure dynamically. Additionally, the repeated use of attention mechanisms in the method contributes to the increased computational complexity of MDHGC. However, in terms of classification accuracy, particularly in complex scenarios, the performance of MDHGC is clearly superior to other methods. Therefore, for scenarios where computational resources are relatively abundant and high accuracy is crucial, MDHGC provides an attractive option.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:26:46 UTC from IEEE Xplore. Restrictions apply.

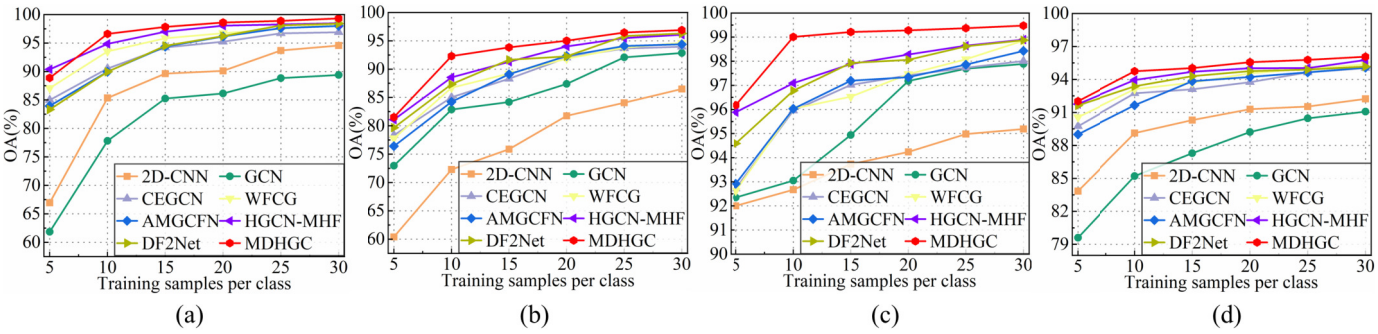


Fig. 11. OA of each model with different quantities of training labeled samples for each class. (a) PU. (b) IP. (c) SA. (d) LN.

TABLE XI
COMPUTATIONAL COMPLEXITY OF DIFFERENT METHODS ON DIFFERENT DATASETS

		2D-CNN	GCN	Base	CEGCN	WFCG	AMGCFN	DF2Net	HGCN-MHF	MDHGC
PU	T(s)	16.45	31.45	26.49	54.18	69.17	109.56	94.18	150.26	244.13
	GFP	0.37	1.11	0.78	10.96	13.23	14.98	10.39	15.71	23.02
	Par	42851	82245	70672	152857	196781	343614	163400	167188	501032
IP	T(s)	5.38	10.60	9.27	19.75	22.39	29.52	26.41	37.60	68.28
	GFP	0.29	0.81	0.51	1.31	1.62	2.05	1.53	2.77	5.17
	Par	56712	83136	74726	166370	197811	368217	176271	184546	546322
SA	T(s)	12.53	24.75	22.68	35.04	57.09	78.58	69.02	99.62	175.76
	GFP	0.32	1.02	0.54	7.58	8.62	10.81	10.42	13.58	28.57
	Par	56882	84255	76416	166890	198386	369009	178952	185066	564986
LN	T(s)	15.68	32.81	29.60	82.07	144.82	172.75	154.78	228.31	436.15
	GFP	0.72	2.40	2.18	29.37	34.31	39.48	23.20	50.83	97.62
	Par	48382	82754	72904	161174	196878	356859	178426	184342	511500

V. CONCLUSION

In this article, a novel multifeature collaborative attention dynamic hypergraph convolutional network for HSIC is proposed, aimed at flexibly mining higher order interrelationships in HSI. This method introduces a superpixel branch to flexibly and robustly explore the high-order topological structure information of HSI. First, we introduce two differential attention modules to help the hypergraph focus on key vertices of the graph. Then, we fully utilize vertex information to construct a static–dynamic collaborative multiview hypergraph convolutional network, enabling the effective extraction of superpixel-level features. Furthermore, the method designs a pixel-level branch to integrate multiscale, global–local features to complement fine-grained characteristics. Finally, through weighted fusion, the method fully leverages features from different layers, enhancing the feature extraction ability and classification performance for HSI. Compared to existing methods, MDHGC demonstrates superior performance. We hope that this work will inspire further research into hypergraph neural networks. However, MDHGC exhibits high computational complexity. In the future, we will continue to explore an unsupervised, lightweight hypergraph framework to enable more computationally efficient HSI classification.

ACKNOWLEDGMENT

The numerical calculations in this paper have been done on the supercomputing system in the Supercomputing Center of Wuhan University.

REFERENCES

[1] Y. Dong, Z. Yang, Q. Liu, R. Zuo, and Z. Wang, "Fusion of GaoFen-5 and Sentinel-2B data for lithological mapping using vision transformer dynamic graph convolutional network," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 129, May 2024, Art. no. 103780.

[2] X. Ding and W. Hu, "Advancements in geological disaster monitoring and early warning systems: A deep learning and computer vision approach," *Traitement du Signal*, vol. 40, no. 3, pp. 1195–1202, Jun. 2023.

[3] N. Zhang et al., "Progress and prospects of hyperspectral remote sensing technology for crop diseases and pests," *Nat. Remote Sens. Bull.*, vol. 25, no. 1, pp. 403–422, 2021.

[4] M. A. Calin, S. V. Parasca, D. Savastru, and D. Manea, "Hyperspectral imaging in the medical field: Present and future," *Appl. Spectrosc. Rev.*, vol. 49, no. 6, pp. 435–447, Aug. 2014.

[5] Y. Dong, B. Zhu, X. Yang, and X. Ma, "Deep metric learning based on Brownian covariance representation for few-shot hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5507813.

[6] B. Qin, S. Feng, C. Zhao, B. Xi, W. Li, and R. Tao, "FDGNet: Frequency disentanglement and data geometry for domain generalization in cross-scene hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 6, pp. 10297–10310, Jun. 2025.

[7] S. Shanmugam and P. SrinivasaPerumal, "Spectral matching approaches in hyperspectral image processing," *Int. J. Remote Sens.*, vol. 35, no. 24, pp. 8217–8251, Dec. 2014.

[8] L. Wei, H. Ma, Y. Yin, and C. Geng, "Kmeans-CM algorithm with spectral angle mapper for hyperspectral image classification," *IEEE Access*, vol. 11, pp. 26566–26576, 2023.

[9] J. Peng, L. Li, and Y. Y. Tang, "Maximum likelihood estimation-based joint sparse representation for the classification of hyperspectral remote sensing images," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 6, pp. 1790–1802, Jun. 2019.

[10] W. Zhao and S. Du, "Spectral–spatial feature extraction for hyperspectral image classification: A dimension reduction and deep learning approach," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 8, pp. 4544–4554, Aug. 2016.

[11] V. Bruni, G. Maiello, G. Monteverdea, A. Paglialungab, and D. Vitulanoa, "A wavelet-based band selection method for hyperspectral image classification," in *Proc. 13th Workshop Hyperspectral Imag. Signal Process., Evol. Remote Sens. (WHISPERS)*, Oct. 2023, pp. 1–5.

[12] F. Tsai and J.-S. Lai, "Feature extraction of hyperspectral image cubes using three-dimensional gray-level cooccurrence," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 6, pp. 3504–3513, Jun. 2013.

[13] B. Zhu, Y. Jin, X. Guan, and Y. Dong, "SSMM: Semi-supervised manifold method with spatial–spectral self-training and regularized metric constraints for hyperspectral image dimensionality reduction," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 136, Feb. 2025, Art. no. 104373.

[14] J. Peng, Y. Zhou, and C. L. P. Chen, "Region-kernel-based support vector machines for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 9, pp. 4810–4824, Sep. 2015.

[15] A. Merentitis, C. Debes, and R. Heremans, "Ensemble learning in hyperspectral image classification: Toward selecting a favorable bias-variance tradeoff," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 4, pp. 1089–1102, Apr. 2014.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:26:46 UTC from IEEE Xplore. Restrictions apply.

[16] X. Qin, Y. Zhang, and Y. Dong, "Domain alignment dynamic spectral and spatial feature fusion for hyperspectral change detection," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 18, pp. 557–568, 2025.

[17] X. Huang, Y. Zhang, F. Luo, and Y. Dong, "Dynamic token augmentation mamba for cross-scene classification of hyperspectral image," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5539713.

[18] C. Liu, X. Ma, X. Yang, Y. Zhang, and Y. Dong, "COMO: Cross-Mamba interaction and offset-guided fusion for multimodal object detection," 2024, *arXiv:2412.18076*.

[19] M. Wang, S. Lu, D. Zhu, J. Lin, and Z. Wang, "A high-speed and low-complexity architecture for softmax function in deep learning," in *Proc. IEEE Asia–Pacific Conf. Circuits Syst. (APCCAS)*, Oct. 2018, pp. 223–226.

[20] Y. Duan, F. Luo, M. Fu, Y. Niu, and X. Gong, "Classification via structure-preserved hypergraph convolution network for hyperspectral image," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5507113.

[21] H. Lee and H. Kwon, "Going deeper with contextual CNN for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 26, no. 10, pp. 4843–4855, Oct. 2017.

[22] Y. Li, H. Zhang, and Q. Shen, "Spectral–spatial classification of hyperspectral imagery with 3D convolutional neural network," *Remote Sens.*, vol. 9, no. 1, p. 67, Jan. 2017.

[23] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3D-2D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2020.

[24] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[25] X. Zhang, S. Shang, X. Tang, J. Feng, and L. Jiao, "Spectral partitioning residual network with spatial attention mechanism for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5507714.

[26] J. Yang, C. Wu, B. Du, and L. Zhang, "Enhanced multiscale feature fusion network for HSI classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 12, pp. 10328–10347, Dec. 2021.

[27] A. Qin, Z. Shang, J. Tian, Y. Wang, T. Zhang, and Y. Y. Tang, "Spectral–spatial graph convolutional networks for semisupervised hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 2, pp. 241–245, Feb. 2019.

[28] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2021.

[29] W. Li, Q. Liu, S. Fan, H. Bai, and M. Xin, "Multistage superpixel-guided hyperspectral image classification with sparse graph attention networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5519718.

[30] S. Wan, C. Gong, P. Zhong, B. Du, L. Zhang, and J. Yang, "Multiscale dynamic graph convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3162–3177, May 2020.

[31] J. Lu and S.-I. Kamata, "Multiscale superpixel HGCN combining CNN for semi-supervised hyperspectral image classification," *J. Adv. Inf. Technol.*, vol. 15, no. 8, pp. 991–1000, 2024.

[32] M.-Y. Liu, O. Tuzel, S. Ramalingam, and R. Chellappa, "Entropy rate superpixel segmentation," in *Proc. CVPR*, Jun. 2011, pp. 2097–2104.

[33] Q. Liu, L. Xiao, J. Yang, and Z. Wei, "CNN-enhanced graph convolutional network with pixel- and superpixel-level feature fusion for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 10, pp. 8657–8671, Oct. 2021.

[34] Y. Dong, Q. Liu, B. Du, and L. Zhang, "Weighted feature fusion of convolutional neural network and graph attention network for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 31, pp. 1559–1572, 2022.

[35] H. Zhou, F. Luo, H. Zhuang, Z. Weng, X. Gong, and Z. Lin, "Attention multihop graph and multiscale convolutional fusion network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5508614.

[36] X. Zhu, D. Wang, J. Li, R. Su, Q. Wan, and Y. Zhou, "Dynamical attention hypergraph convolutional network for group activity recognition," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 5, pp. 8911–8925, May 2025.

[37] Q. Xu, S. Xu, J. Liu, and L. Huang, "Dynamic hypergraph convolution and recursive gated convolution fusion network for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 20, pp. 1–5, 2023.

[38] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao, "Hypergraph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 3558–3565.

[39] Z. Ma, Z. Jiang, and H. Zhang, "Hyperspectral image classification using feature fusion hypergraph convolution neural network," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5517314.

[40] Y. Wang, Z. Xue, M. Jia, Z. Liu, and H. Su, "Hypergraph convolutional network with multiple hyperedges fusion for hyperspectral image classification under limited samples," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5526318.

[41] Q. Wang, J. Huang, P. Jin, Y. Gu, and T. Shen, "Differential feature-enhanced fusion network for hyperspectral image classification," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2024, pp. 9104–9108.

[42] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Süsstrunk, "SLIC superpixels compared to state-of-the-art superpixel methods," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 11, pp. 2274–2282, Nov. 2012.

[43] W. Xue, L. Zhang, X. Mou, and A. C. Bovik, "Gradient magnitude similarity deviation: A highly efficient perceptual image quality index," *IEEE Trans. Image Process.*, vol. 23, no. 2, pp. 684–695, Feb. 2014.

[44] M. Pietikäinen, "Local binary patterns," *Scholarpedia*, vol. 5, no. 3, p. 9775, 2010.

[45] R. O. Brown and D. I. A. MacLeod, "Color appearance depends on the variance of surround colors," *Current Biol.*, vol. 7, no. 11, pp. 844–849, Nov. 1997.

[46] J. Jiang, Y. Wei, Y. Feng, J. Cao, and Y. Gao, "Dynamic hypergraph neural networks," in *Proc. IJCAI*, 2019, pp. 2635–2641.

[47] Y. Chang, Q. Liu, Y. Zhang, and Y. Dong, "Unsupervised multiview graph contrastive feature learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5524314.

[48] S. Jia, B. Deng, J. Zhu, X. Jia, and Q. Li, "Local binary pattern-based hyperspectral image classification with superpixel guidance," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 749–759, Feb. 2018.

[49] I. B. Barcelos, F. D. C. Belém, L. D. M. João, Z. K. G. D. Patrocínio, A. X. Falcão, and S. J. F. Guimarães, "A comprehensive review and new taxonomy on superpixel segmentation," *ACM Comput. Surv.*, vol. 56, no. 8, pp. 1–39, Aug. 2024.

[50] M. Wang, X. Liu, Y. Gao, X. Ma, and N. Q. Soomro, "Superpixel segmentation: A benchmark," *Signal Process., Image Commun.*, vol. 56, pp. 28–39, Aug. 2017.

[51] S. Subudhi, R. N. Patro, P. K. Biswal, and F. Dell'Acqua, "A survey on superpixel segmentation as a preprocessing step in hyperspectral image analysis," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 5015–5035, 2021.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:26:46 UTC from IEEE Xplore. Restrictions apply.